Günter Böckle

1

Introduction to Software Product Line Engineering

In this chapter you will learn:

- *The key principles of product line engineering for mechanical and electronic consumer products.*
- *The key ideas of the software product line engineering paradigm.*
- *The motivations and the prerequisites for adapting those principles to software engineering.*

1.1 Principles of Product Line Engineering

Production line

The way that goods are produced has changed significantly in the course of time. Formerly goods were handcrafted for individual customers. By and by, the number of people who could afford to buy various kinds of products increased. In the domain of automobiles this led to Ford's invention of the *production line*, which enabled production for a mass market much more cheaply than individual product creation on a handcrafted basis. However, the production line reduced the possibilities for diversification.

Individual and standard software

Roughly, both types of products, individual and mass produced ones can be identified in the software domain as well: they are denoted as individual software and standard software. Generally, each of these types of products has its drawbacks. Individual software products are rather expensive, while standard software products lack sufficient diversification.[2]

1.1.1 Mass Customisation

Individualised products

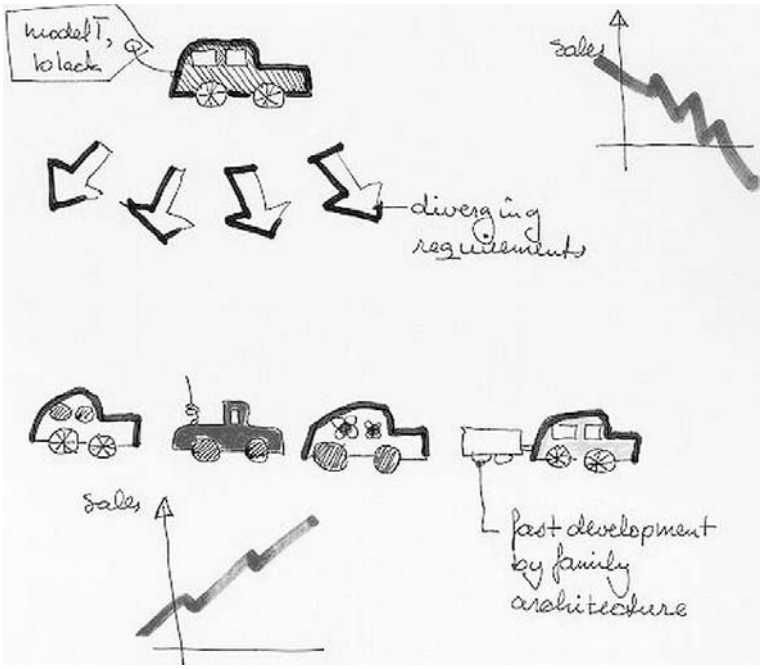
Customers were content with standardised mass products for a while – but not all people want the same kind of car for any purpose. Certain cars are used for travelling by a single person others for big families. Some cars are used by people living in cities, others mainly in the countryside. People want to have another car or a better one than their neighbours. Thus, industry was confronted with a rising demand for individualised products. This was the beginning of *mass customisation*, which meant taking into account the customers' requirements and giving them what they wanted (see Fig. 1-1).

We use the following definition of mass customisation:

> ***Definition 1-1:*** *Mass Customisation*
>
> Mass customisation is the large-scale production of goods tailored to individual customers' needs.
>
> [Davis 1987]

> ***Example 1-1:*** *Mass Customisation in Car Manufacturing*
>
> The four cars at the bottom of Fig. 1-1 are an example of mass customisation: different cars that meet individual customers' wishes, each produced in large numbers.

[2] See [Halmans and Pohl 2002] for a treatment of product line engineering for individual vs. mass-market software.



Not "one product fits all"

Fig. 1-1: Single product for all vs. individual products[3]

1.1.2 Platforms

Prerequisite for mass customisation

For the customer mass customisation means the ability to have an individualised product. For the company mass customisation means higher technological investments which leads to higher prices for the individualised products and/or to lower profit margins for the company. Both effects are undesirable. Thus many companies, especially in the car industry, started to introduce *common platforms* for their different types of cars by planning beforehand which parts will be used in different car types.

Originally, an automobile platform only consisted of floor panels, a suspension system, and rocker panels. Later more parts were added to the platform. The platform provided a structure for major components determining the body size and the size and type of the engine and transmission. The parts comprising the platform were usually the most expensive subsystem in terms of design and manufacturing preparation costs. The use of the platform for different car types typically led to a reduction in the production cost for a particular car type.

[3] Picture drawn by Loe Feijs.

Platform: 35% sales growth

The platform approach enabled car manufacturers to offer a larger variety of products and to reduce costs at the same time. The result was that within periods of three years (measured from 1980 to 1991 and averaged) those companies using the best platform strategy increased sales by 35%, whereas those companies starting from scratch for each new series of cars had a sales loss of 7% (for details, see [Cusumano and Nobeoka 1998]). The same pattern was observable in the camera industry and many others (Examples 1-2 and 1-3).

Example 1-2: *From the Camera World*

In 1987, Fuji released the Quicksnap, the first single-use camera. It caught Kodak by surprise: Kodak had no such product in a market that grew from then on by 50% annually, from 3 million in 1988 to 43 million in 1994. However, Kodak won back market share and in 1994, it had conquered 70% of the US market. How did Kodak achieve it? First, a series of clearly distinguishable, different camera models was built based on a common platform. Between April 1989 and July 1990, Kodak reconstructed its standard model and created three additional models, all with common components and the same manufacturing process. Thus, Kodak could develop the cameras faster and with lower costs. The different models appealed to different customer groups. Kodak soon had twice as many models as Fuji, conquered shelf space in the shops and finally won significant market share this way (for details see [Robertson and Ulrich 1999; Clark and Wheelwright 1995]).

Example 1-3: *Notion of Platform in Office Supplies*

The "Post-It" notes from 3M are considered a platform from which many individual products have been derived. For instance, "Post-it" notes with a company logo, or markers to select pages in books, etc. [Cooper et al. 2001].

Definition 1-2: *Platform*

A platform is any base of technologies on which other technologies or processes are built.

[TechTarget 2004]

"Platform" used in various contexts

The term platform is used in various contexts as illustrated by Examples 1-1 to 1-3. A common definition of platform does not exist. For this book we use the definition given in Definition 1-2. Note that this definition encompasses

all kinds of reusable artefacts as well as all kinds of technological capabilities.

1.1.3 Combining Platform-Based Development and Mass Customisation

Platform and mass customisation

The combination of mass customisation and a common platform allows us to reuse a common base of technology and, at the same time, to bring out products in close accordance with customers' wishes. The systematic combination of mass customisation and the use of a common platform for the development of software-intensive systems and software products is the key focus of this book. We call the resulting software development paradigm *software product line engineering*. In Section 1.2 we sketch the key implications of combining those approaches. The motivations behind the product line engineering paradigm are outlined in Section 1.3.

1.2 Engineering Customised Products

Combining platform and mass customisation in order to provide customised products has many implications for the development process as well as the developing organisation. We briefly sketch the three key influences.

1.2.1 Creating the Platform

Preparing for mass customisation

In single-system engineering, products are regarded as independent, self-contained items. In the example presented in Fig. 1-1 (bottom), this means having four distinct projects for developing four distinct products. Developing these four types of cars by product line engineering requires the creation of a platform that suits all of them. This platform comprises all common parts, for instance a gear box that can be applied in all or most of the cars (in so far, the platform extends the original meaning of a car platform as described in Section 1.1.2). Furthermore, the platform determines the specifics that distinguish not only the four cars but also how to accommodate the customers' wishes for more individualised products. Briefly, creating the platform implies preparing for mass customisation. For our example, this means having four basic types of cars, each with numerous customisable features.

Commonality first, differences later

The strategy that we have intuitively followed in the above example is, first, to focus on what is common to all products, and next, to focus on what is different. In the first step, artefacts are provided that can be reused for all products. These artefacts may be built from scratch or derived from another platform or earlier systems. Built-in flexibility makes it possible to reuse these artefacts in different applications, providing mass customisation. Cre-

ating this flexibility requires some effort. Therefore, in the automobile industry, the platform is usually developed by a distinct project. Several other projects are commissioned to develop the single cars or, more frequently, groups of cars, each group encompassing a subset of the product line's products.

1.2.2 Introducing Flexibility

Flexibility is the key

To facilitate mass customisation, the artefacts used in different products have to be sufficiently adaptable to fit into the different systems produced in the product line. This means that throughout the development process we have to identify and describe where the products of the product line may differ in terms of the features they provide, the requirements they fulfil, or even in terms of the underlying architecture etc. We thus have to provide flexibility in all those artefacts to support mass customisation.

Different cars of the same product line may for instance have different windshield wipers and washers. We design the cars in a way that allows a common approach to support the different motors for these different windshield wipers/washers, their different sizes, etc. Such flexibility comes with a set of constraints. If you drive a convertible, you would not want a rear window washer splashing water onto the seats! Therefore, the selection of a convertible car means the flexibility that the availability of the windshield wipers and washers is restricted, so that the rear window washer is disabled when the car roof is open.

This flexibility is a precondition for mass customisation; it also means that we can predefine what possible realisations shall be developed (there are only a certain number of windshield wiper configurations conceivable). In addition, it means that we define exactly the places where the products can differ so that they can have as much in common as possible.

Variability

The flexibility described here is called "variability" in the software product-line context. This variability is the basis for mass customisation. Variability is introduced and defined in detail in Part II of this book.

Sharing platforms between product lines

In the automotive industry, cars sharing the same platform and exhibiting similar features were called a *product line* or a *product family*. At this stage, the relation between a platform and a product line was straightforward. A manageable set of cars belonged to one product line that was based on a single platform. But soon, in the history of automobile development, platforms were shared across different product lines. New product lines used platforms of earlier product lines etc. Thus, the simple relation between platform and product line vanished. Consequently, it became necessary to manage carefully the trace information from a platform to the products derived from it. Without such trace information, it is barely possible to find out

which parts of the platform have been used in which product. Thus, for example, the estimation of the impact of changes becomes difficult.

1.2.3 Reorganising the Company

A platform leads to standardisation

Migrating from single-system engineering to a platform approach has far-reaching consequences. The products derived from platform artefacts can no longer be treated as being independent. They are related through the underlying technology. This also has to be reflected in the *organisation structure* of a company: it may be necessary to establish additional *organisation units*, e.g. one unit responsible for the platform, or by setting up additional communication paths between formerly independent units. Basically, the platform approach leads to *standardisation* of procedures, workflows, and the technology employed within an organisation or even across organisations.

1.3 Motivations for Product Line Engineering

We already conveyed a first idea of the goals that product line engineering pursues: to provide customised products at reasonable costs. In this section, we briefly outline the key motivations for developing software under the product line engineering paradigm.

1.3.1 Reduction of Development Costs

Up-front investment

A far-reaching change of engineering practices is usually not initiated without sound economical justification. An essential reason for introducing product line engineering is the reduction of costs. When artefacts from the platform are reused in several different kinds of systems, this implies a cost reduction for each system. Before the artefacts can be reused, investments are necessary for creating them. In addition the way in which they shall be reused has to be planned beforehand to provide managed reuse. This means that the company has to make an up-front investment to create the platform before it can reduce the costs per product by reusing platform artefacts.

Pay-off around three systems

Figure 1-2 shows the *accumulated costs* needed to develop *n* different systems. The solid line sketches the costs of developing the systems independently, while the dashed line shows the costs for product line engineering. In the case of a few systems, the costs for product line engineering are relatively high, whereas they are significantly lower for larger quantities. The location at which both curves intersect marks the *break-even point*. At this point, the costs are the same for developing the systems separately as for developing them by product line engineering. Empirical investigations revealed that,

for software, the break-even point is already reached around three systems.[4] A similar figure is shown in [Weiss and Lai 1999], where the break-even point is located between three and four systems. The precise location of the break-even point depends on various characteristics of the organisation and the market it has envisaged, such as the customer base, the expertise, and the range and kinds of products. The strategy that is used to initiate a product line also influences the break-even point significantly [McGregor et al. 2002]. Chapter 20 elaborates on the initiation of product lines.

Lower development cost

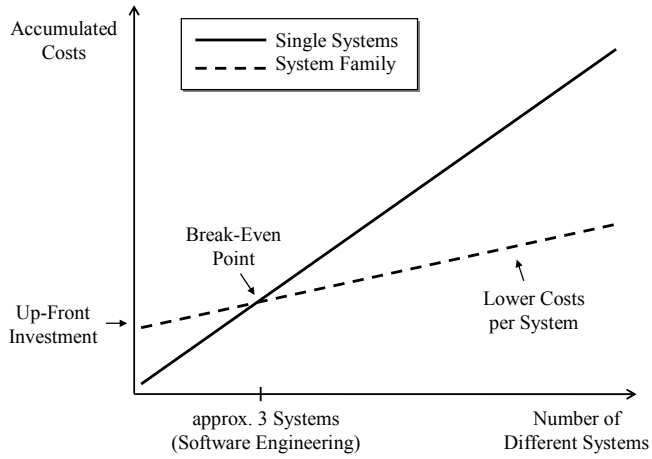


Fig. 1-2: Costs for developing n kinds of systems as single systems compared to product line engineering

1.3.2 Enhancement of Quality

Improved quality through reuse

The artefacts in the platform are reviewed and tested in many products. They have to prove their proper functioning in more than one kind of product. The extensive quality assurance implies a significantly higher chance of detecting faults and correcting them, thereby increasing the quality of all products.

1.3.3 Reduction of Time to Market

Shorter development cycles

Often, a very critical success factor for a product is the time to market. For single-product development, we assume it is roughly constant,[5] mostly comprising the time to develop the product. For product line engineering, the time to market indeed is initially higher, as the common artefacts have to be built first. Yet, after having passed this hurdle, the time to market is consid-

[4] [Clements and Northrop 2001]: The sidebar on p. 226, "It Takes Two", provides a closer examination of the break-even point for software product lines.

[5] In practice, this number varies, but for showing the effect of single-system vs. product line engineering this assumption is sufficiently accurate.

erably shortened as many artefacts can be reused for each new product (see Fig. 1-3).

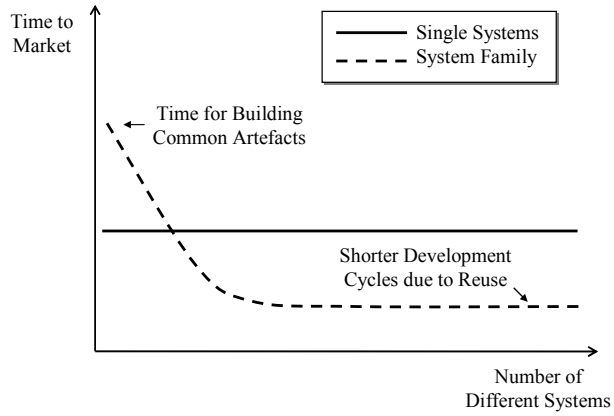


Fig. 1-3: Time to market with and without product line engineering

1.3.4 Additional Motivations

In this section, we briefly outline additional motivations for introducing a software product line engineering paradigm. An overview of further motivations (and also key principles and solutions) can be found in [Pohl et al. 2001b].

1.3.4.1 Reduction of Maintenance Effort

Propagation of error corrections

Whenever an artefact from the platform is changed, e.g. for the purpose of error correction, the changes can be propagated to all products in which the artefact is being used. This may be exploited to reduce maintenance effort. At best, maintenance staff do not need to know all specific products and their parts, thus also reducing learning effort. However, given the fact that platform artefacts are changed, testing the products is still unavoidable. Yet, the reuse of test procedures is within the focus of product line engineering as well and helps reduce maintenance effort. The techniques used in product line engineering make a system better maintainable as stated in [Coplien 1998]: “The same design techniques that lead to good reuse also lead to extensibility and maintainability over time.”

1.3.4.2 Coping with Evolution

Organised evolution

The introduction of a new artefact into the platform (or the change of an existing one) gives the opportunity for the evolution of all kinds of products derived from the platform. Similarly, developers who want to introduce a trend towards certain product features may do so by adding specific artefacts

to the platform. Thus, it is possible to better organise development for evolution of the product range and reduce the effort compared to single-system engineering.

1.3.4.3 Coping with Complexity

Complexity by size increase

Due to the increased number of customers' wishes, the complexity of products increases. This holds especially for software, where code size and complexity sometimes increase beyond being manageable. The reason for this trend is that more and more functionality is put into software.

> ***Example 1-4:*** *Size of the Windows Operating System*
>
> In 1991 the size of Windows NT PDK 2 was 1.8 million SLOC,[6] see [Cusumano and Selby 1998] – Windows XP has about 45 million SLOC.

Complexity by hardware–software migration

In embedded systems, the complexity increase was further aggravated by moving functionality from hardware to software. This was done for several reasons, mostly because software provides more flexibility than hardware as well as a higher level of abstraction. Software enables complex interactions between functionality, and allows for distribution over a network. This causes a significant increase in the complexity of the system, making it difficult for developers to conceive the whole functionality. If no adequate measures are taken, high complexity leads to rapidly increasing error rates, long development cycles, and a higher time to market – influencing our key motivations.

Platform reduces complexity

The fact that the common parts are reused throughout the product line reduces complexity significantly. The platform provides a structure that determines which components can be reused at what places by defining variability at distinct locations; this reduces complexity. The reuse of common parts from the platform reduces the error rate and the development time.

1.3.4.4 Improving Cost Estimation

Platform simplifies cost estimation

The development organisation can focus its marketing efforts on those products that it can easily produce within the product line. Nevertheless, it can allow extensions not covered by the platform. Products that do need such extensions can be sold for higher prices than those products built by reusing platform artefacts only. Furthermore, calculating prices for products realised within the product line is relatively straightforward and does not include much risk. Consequently, the platform provides a sound basis for cost estimation.

---

[6] SLOC = Source Lines of Code, the number of non-comment lines of code in a source program.

1.3.4.5 Benefits for the Customers

Customers get higher quality for lower prices

Customers get products adapted to their needs and wishes. This is just what they ask for – previously, users had to adapt their own way of working to the software. In the past, it often happened that customers had to get used to a different user interface and a different installation procedure with each new product. This annoyed them, in particular as it even happened when replacing one version of a product by the next version. So customers began to ask for improved software ergonomics. Accordingly, software packages were developed to support common user interfaces and common installation procedures. The use of such packages contributed to the proliferation of the idea of platforms. Moreover, customers can purchase these products at a reasonable price as product line engineering helps to reduce the production costs. Additionally, customers get higher quality products since the reusable components and their configurations have been tested in many products developed earlier and proved dependable. Moreover, many requirements are reviewed more often, not only in domain engineering but also in application engineering, and the architecture review for application architectures helps to find problems in the domain architecture, too.

Common look and feel

Despite possessing individual features, the products of a product line have a lot in common due to the artefacts reused from the platform. Similar or the same user interfaces and similar major functionality make it easy for the customer to switch from one product to another. The customer does not have to learn new ways of using another product derived from the same platform.

1.4 Software Product Line Engineering

Past situation on software market

Until recently software, especially embedded software was relatively small[7] and each product variant got its own variant of software. Software was a way to easily implement on the same hardware individual product variants that originally required individual hardware variants. Compared to hardware, software is easy and cheap to copy, transport, and replace. This fact was exploited by employing software flexibly and adapting it at very late stages of development, thus easing many problems of system developers. Not much thought was spent on how software was produced. Outside the embedded system world, software was typically not regarded as being variable. Either a customer could buy a software system including all possible features one might ever need, or software was produced for a single purpose by order of a single customer.

[7] See the introduction of [V. Ommering 2004], where it is shown how TV-set software grew from 1 kB in 1978 to 100 kB in 1990 and to 10,000 kB in 2004.

Demand for product line engineering

However, the situation in software engineering has changed. Almost all systems of a certain complexity contain software. Many systems in our environment are becoming software-intensive systems, not only because variability can be implemented more flexibly than in pure hardware, but also because of the fact that software allows the introduction of new functionality that could not easily be achieved without it. The amount of embedded software is growing, and the amount of variability is growing even faster. In many systems, this amount is now many times larger than before software came into play. Therefore, presently a strong need for adopting product line engineering can be observed in the software domain, especially when size and complexity exceed the limits of what is feasible with traditional approaches.

1.4.1 Definition

We define the term software product line engineering as follows:

> ***Definition 1-3:*** *Software Product Line Engineering*
>
> Software product line engineering is a paradigm to develop software applications (software-intensive systems and software products) using platforms and mass customisation.

Pure and embedded software

Our definition covers the development of pure software products or systems as well as the development of software that is embedded into a software-intensive system, i.e. a system that closely integrates hardware and software. Being an integral part of many everyday products, embedded software constitutes a great proportion of the total amount of software being developed and sold. With respect to this book, it is valid to consider both kinds of software, as the principles of product line engineering are the same for them.

Managed variability

Developing applications using *platforms* means to plan proactively for reuse, to build reusable parts, and to reuse what has been built for reuse. Building applications for *mass customisation* means employing the concept of *managed variability*, i.e. the commonalities and the differences in the applications (in terms of requirements, architecture, components, and test artefacts) of the product line have to be modelled in a common way. The definition of the commonalities and the variability, the documentation, the subsequent deployment, and the management are key focuses of this book.

Constraining adaptations

Managed variability has a great impact on the way software is developed, extended, and maintained. Usually, for those who understand how a piece of software works, it is also easy to change it and adapt it to suit a new purpose. However, such changes often corrupt the original structure of the software and hamper quality aspects like understandability or maintainability. In order

to be able to deal with adaptations in a managed way, they have to be accomplished in a reproducible manner. The abundance of possibilities for adapting a piece of software has to be restricted to those places where it makes sense to do so. More than other engineering disciplines, software product line engineering deals with ways to restrict variation in a manageable way.

1.4.2 Software Platform

Classic computer platform

Definition 1-3 makes use of the term platform. In the software industry, this term is often used to denote the underlying computer system on which application programs can run. This may be the hardware used, e.g. the processor, or, more often, the combination of hardware and operating system (see Example 1-5).

> ***Example 1-5:*** *Symbian as a Platform*
>
> The Symbian operating system is used by several companies as a platform for some of their product lines of mobile phones, including Nokia, Siemens, and Motorola.

Platforms in software architecture

In software architecture the term platform is used slightly differently but still in line with the classic meaning. A software architecture usually consists of multiple layers. From the viewpoint of one layer, its underlying layer is called its platform.

Meaning of platform in this book

Though the above interpretations of platform are quite common in software engineering, they do not reflect what makes up a platform in software product line engineering. This kind of software platform must enable the creation of entire products from reusable parts. Therefore, we use the definition given in Definition 1-4.

> ***Definition 1-4:*** *Software Platform*
>
> A software platform is a set of software subsystems and interfaces that form a common structure from which a set of derivative products can be efficiently developed and produced.
>
> [Meyer and Lehnerd 1997]

The subsystems belonging to a software platform encompass not just code but also requirements, architecture, and other artefacts of the development process.

1.4.3 Prerequisites

The fact that product line engineering entered the software engineering domain much later[8] than other domains was due to several reasons. Past barriers for adapting the product line engineering paradigm to software development have been partly overcome but can still complicate the adoption of software product line engineering if not taken carefully into consideration. To overcome these barriers, certain *prerequisites* are necessary for adopting software product line engineering.

1.4.3.1 Enabling Technologies

Implementation technology

For a long time, an important barrier to adopting software product line engineering was the lack of adequate technology for applying the principles of product line engineering in an easy way. In addition, part of the technology available was not used in practice. The development of such enabling technology supported the introduction of software product line engineering. Actually, the implementation technologies of standard software engineering are used for product line engineering, no new ones are introduced. Coplien states that "many software families are naturally expressed by existing programming language constructs" [Coplien 1998].

Object-oriented programming

Object-oriented programming is a major enabling technology. Many programs were (and still are) written in procedural programming languages. These languages make encapsulation and information hiding hard to put into practice. Yet, encapsulation is a prerequisite for realising managed variability. Object-oriented modelling and programming concepts mitigated this barrier by supporting approved design principles in a more natural way.

Component technology

Another important achievement is the introduction of component technology. Component technology enables developers to package software as loosely coupled parts. Components can be developed, compiled, linked and loaded separately. Only at run-time are they combined into a working system. This helps developers to focus on a particular component and thus to cope with complexity. Besides, component technology limits the scope where variation is possible and thus supports the realisation of managed variability.

Binding techniques

Late-binding techniques, especially installation and run-time dynamic binding, allow for late configuration choices. By applying late binding, variability can be designed and implemented in a uniform way without bothering about how the actual variants look. This facilitates the implementation of platforms and provides an easy way to realise mass customisation.

[8] However, the basic ideas had already been proposed in the 1970s by Dijkstra [Dijkstra 1972] and Parnas [Parnas 1976].

Middleware

The quasi-standardisation of middleware, interoperability, and interface descriptions supports late binding. It also speeds up the configuration of applications enormously and helps in defining the platform and its boundaries. Software from different origins can be used to work together inside an application. Middleware facilitates the development of loosely coupled components, thus contributing to reduced complexity and easing managed reuse.

Configuration management

Configuration management is another important prerequisite for being able to cope with the complexity of large applications consisting of many parts in different versions. Therefore, sophisticated configuration management is necessary to succeed in software product line engineering.

1.4.3.2 Process Maturity

Lacking maturity in software engineering can affect the success of software product line engineering even more seriously than technological matters.

Process models

For a long time, software development processes were unstructured, hardly well defined, and also not well understood. The increasing application of assessments, e.g. postulated by CMMI led to better software development processes. Process models such as CMMI help to identify the weak parts of software development processes, thus driving their improvement and fostering the use of sound engineering principles in software practice.

Requirements engineering

The CHAOS report [Standish Group 1995] showed that inadequate requirements engineering is a major cause of problems in software projects. Consequently, thorough requirements engineering comprising the identification of commonality and variability is a major prerequisite for software product line engineering.

Modelling techniques

The history of software and system development shows that abstraction plays a major role in making the complexity manageable. Although modelling techniques were available, they were often not used or people used different techniques, mostly on an ad hoc basis. The major driver for using modelling techniques was the foundation of UML [OMG 2003], which has become an industrial standard. This standardisation of modelling notations helps engineers to become aware of methods for modelling variability, too.

1.4.3.3 Domain Characteristics and Expertise

Another major prerequisite for software product line engineering is sufficient domain expertise. Only people who know their markets and/or customers can identify commonalities and variability in an adequate way for developing platforms and variability.

Domain knowledge

Software is flexible and all sorts of variability can be added to it. Wrong choices are, however, costly. The introduction of variability that is not used leads to additional costs for developing it and, at a later stage, removing it

again. Failing to introduce the required variability leads to additional costs for realising common and even variable parts in each product. Moreover, it is important to use an appropriate level of abstraction to define the variability of a product line. Ideally, the variability abstractions match the commonly used domain abstractions. Domain-related abstractions are known to the customers and developers and thus help them to understand the variability of the product line. Not knowing the domain leads to incorrect abstractions with the danger that they are not perceived by the stakeholders, and thus cause wrong choices, which have to be repaired afterwards.

The better you know the domain and the more experience you have with your products, the more likely it is that you can successfully introduce a software product line.

Domain stability

The stability of the domain is also an important factor for the successful introduction of software product line engineering. If everything changes every half-year in an unpredictable way, the investment costs never pay off. This situation is similar to not understanding the domain well: variability is added that is not needed and the variability that is actually required is not available.